

Internship Project Report

Expense Tracker — A Command-Line Application in Python

Submitted by: Samvit Srivastava

Designation: Python Developer Intern (Project Intern)

Organisation: Codec Technologies India Pvt. Ltd., Chandivali IT Hub, Mumbai

Internship duration: 08/07/2026 to 08/08/2026 (1 month)

Mode: Pan India, Hybrid

Programme: 1-Month AICTE & ICAC Approved Internship Programme, listed on the National Internship Portal

AICTE ID: CORPORATE6759d549ce59e1733940553

NCS ID: E19E86-0116588288923

Reporting to: Assigned Project Head(s)

Programme Manager: Dr. Anurag Shrivastava

Repository: <https://github.com/samvit-iilm/python-Internship>

1. Acknowledgement

I would like to thank Codec Technologies India Pvt. Ltd. for offering me the opportunity to work as a Python Developer Intern, and Dr. Anurag Shrivastava and my assigned project head for their guidance during the programme. The internship was conducted under the AICTE and ICAC approved programme listed on the National Internship Portal, and gave me the chance to take a Python project from an idea through to a tested and documented tool.

2. Abstract

Most students track expenses either in their head or in a messy notes app, and neither answers the question that actually matters at the end of the month: *where did the money go?*

Expense Tracker is a command-line application that records expenses and answers that question in one command. Each entry stores an amount, a category, a short note and a date; the tool can then list them, filter by category or month, delete a wrong entry, export everything to CSV, and print a summary showing the total spent, the biggest single expense, and a category-wise breakdown with a simple bar chart.

It is written in **pure Python 3.10+ using only the standard library** — roughly 300 lines of code across four small modules, with 32 tests. Data is kept in a plain JSON file that a person can open and read.

3. Objectives

1. Build a complete, working Python application rather than a set of exercises.
 2. Practise clean structure — keep the logic, the file handling and the printing in separate modules instead of one long script.
 3. Handle user mistakes properly: negative amounts, empty categories, wrong date formats, unknown ids and a corrupt data file.
 4. Learn to build a proper command-line interface with sub-commands using `argparse`.
 5. Write an automated test suite that proves each part works.
 6. Document the project so that someone else can clone it and use it.
-

4. Problem statement

A student wants to know how much they spent this month and on what. The options usually available are:

Approach	Limitation
Remembering it	Unreliable after a few days

Notes app	No totals, no categories, no filtering
Spreadsheet	Works, but slow to open and type into for a single Rs.40 entry
Bank statement	Misses cash, and category names are unreadable
Mobile expense apps	Ads, sign-ups, and the data lives on someone else's server

The gap: a **fast, offline, zero-install tool** where recording an expense is one short command, and where the data stays in a plain file that belongs to the user.

5. Requirement analysis

Functional requirements

ID	Requirement
F1	Add an expense with amount, category, optional note and date
F2	Use today's date automatically when no date is given
F3	List all expenses, with optional filters by category and month
F4	Delete an expense using its id
F5	Show a summary: total, biggest expense, category-wise and month-wise totals
F6	Export expenses to a CSV file
F7	Save all data to a JSON file automatically
F8	Reject invalid input with a clear message and a non-zero exit code

Non-functional requirements

ID	Requirement
N1	No third-party libraries
N2	Data file readable and editable by a human
N3	Logic separated from input/output so it can be tested
N4	Automated tests for every module
N5	Output readable in a normal terminal

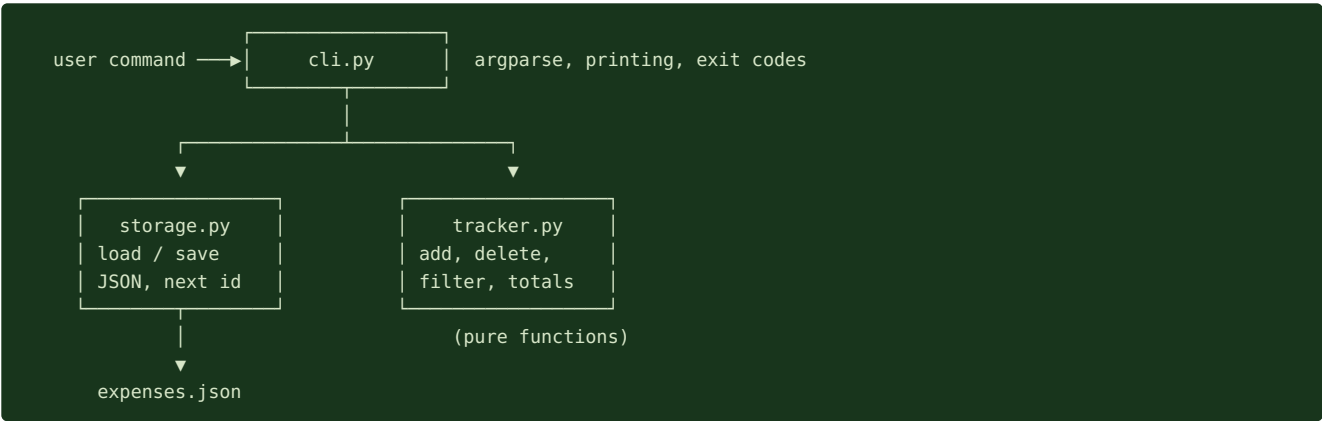
6. Technology used

Component	Choice	Why
Language	Python 3.10+	Required by the internship; modern typing syntax (<code>str</code> <code>None</code>)
Storage	<code>json</code>	Human-readable, built in, no database setup needed
CLI	<code>argparse</code> with sub-commands	Standard help text, type conversion and exit codes for free
Dates	<code>datetime</code>	Validating the <code>YYYY-MM-DD</code> format and getting today's date
Grouping	<code>collections.defaultdict</code>	Clean category-wise and month-wise totals
Export	<code>csv.DictWriter</code>	Correct quoting and escaping without manual string joins
Paths	<code>pathlib.Path</code>	Cleaner and safer than string paths
Testing	<code>unittest</code>	Bundled with Python — keeps the zero-dependency promise

7. System design

7.1 Architecture

The application has four small modules, each with one job:



The important rule: `tracker.py` **never opens a file and never prints anything**. It receives a list, works on it, and returns a value. `storage.py` does all the file work, and `cli.py` does all the printing. This is what makes the logic easy to test — a test can call `add_expense([], 100, "food")` directly without creating any files.

7.2 Module responsibilities

Module	Lines	Responsibility
<code>storage.py</code>	40	Load and save the JSON file, generate the next id
<code>tracker.py</code>	100	Add, delete, filter, totals by category and month, biggest expense
<code>cli.py</code>	150	Sub-commands, table and summary printing, CSV export, exit codes
<code>__main__.py</code>	6	Allows <code>python -m expense_tracker</code>

7.3 Data format

One expense is a dictionary, and the file is a list of them:

```
{
  "id": 1,
  "amount": 6000.0,
  "category": "rent",
  "note": "monthly room rent",
  "date": "2026-07-03"
}
```

Dates are stored as `YYYY-MM-DD` strings on purpose. In that format the text sorts in the same order as the actual dates, and `"2026-08-14".startswith("2026-08")` is all the month filter needs — no date parsing at all while filtering.

8. Implementation highlights

8.1 Validation before saving

Bad data must never reach the file, so `add_expense` checks everything first and raises `ValueError` if anything is wrong:

```
if amount <= 0:
    raise ValueError("amount must be greater than zero")
if not category.strip():
    raise ValueError("category cannot be empty")
```

Categories are also trimmed and lowercased, so `"Food"`, `" food "` and `"FOOD"` all become the same category instead of three separate ones in the summary.

8.2 Reusing deleted ids

A counter that only goes up gives ugly ids like #47 in a list of five items. Instead the tool takes the smallest number that is not currently in use:

```
def next_id(expenses):
    used = {e["id"] for e in expenses}
    candidate = 1
    while candidate in used:
        candidate += 1
    return candidate
```

Deleting #2 from three expenses and adding a new one gives it #2 again.

8.3 Grouping with defaultdict

Category and month totals are the same three lines, which keeps the summary code short:

```
totals = defaultdict(float)
for expense in expenses:
    totals[expense["category"]] += expense["amount"]
ordered = sorted(totals.items(), key=lambda kv: kv[1], reverse=True)
```

Sorting by value puts the biggest spending category at the top, which is the first thing anyone looks for.

8.4 A bar chart with no libraries

The percentage share of each category is drawn by repeating a character:

```
bar = "#" * max(1, round(value / peak * 20))
```

Every category is measured against the largest one, so the longest bar is always 20 characters wide. `max(1, ...)` guarantees that a very small category still shows a visible mark instead of an empty line.

8.5 Errors the user can understand

Every failure prints a plain sentence and returns exit code 1, so the tool also behaves correctly inside a shell script:

```
$ python -m expense_tracker delete 42
error: no expense with id 42

$ python -m expense_tracker add -5 food
error: amount must be greater than zero
```

A corrupt data file is caught in the same way rather than showing a Python traceback:

```
except json.JSONDecodeError:
    raise ValueError(f"{path} is not a valid JSON file")
```

9. Testing

The suite is run with:

```
python -m unittest discover -s tests -v
```

Result: 32 tests, 32 passed, 0 failed.

Test module	Tests	Coverage focus
test_tracker.py	19	Adding, default date, lowercasing, id reuse, all four validation errors, delete, filters by category/month/both, sorting, totals, category and month grouping, biggest expense, empty lists
test_storage_and_cli.py	13	Missing file, save/load round trip, corrupt JSON, and every CLI command end to end including exit codes and the CSV export

Tests use `tempfile.TemporaryDirectory()` so that no test ever touches the real `expenses.json`, and each test cleans up after itself.

Edge cases were tested deliberately, not just the happy path: an empty expense list (total must be `0`, biggest must be `None`), a deleted id being reused, and a filter that matches nothing.

10. Results

Running the tool on the bundled sample data:

```
$ python -m expense_tracker --file sample_expenses.json summary

Total spent : Rs.22,621.83
Entries     : 28
Biggest     : Rs.6,000.00 on rent (2026-07-03)

BY CATEGORY
-----
rent        Rs.12,000.00  53.0% #####
recharge    Rs.2,707.00   12.0% #####
food        Rs.2,470.28   10.9% #####
books       Rs.1,771.87    7.8%   ###
movies      Rs.1,628.84    7.2%   ###
travel      Rs.1,363.40    6.0%   ##
stationery   Rs.680.44      3.0%   #

BY MONTH
-----
2026-07      Rs.12,172.10
2026-08      Rs.10,449.73
```

All eight functional requirements work as specified. The summary immediately shows that rent takes more than half the budget — exactly the kind of answer the tool was built to give. Full output is saved in `docs/sample_output.txt`.

11. Challenges faced

Challenge	Resolution
Same category typed differently (Food , food , FOOD)	Trim and lowercase the category before saving
Ids growing large after deletions	Reuse the smallest free id instead of a counter
Filtering by month without date parsing	Store dates as YYYY-MM-DD text and use <code>startswith("2026-08")</code>
Tests overwriting the real data file	Give every test a <code>tempfile.TemporaryDirectory()</code> of its own
Ugly Python traceback on a corrupt JSON file	Catch <code>JSONDecodeError</code> and raise a readable <code>ValueError</code>
Money printing with wrong decimals	Round on save and format with <code>f"{value:,.2f}"</code> when printing
Bar chart invisible for small categories	<code>max(1, ...)</code> so every category gets at least one mark

12. Learning outcomes

- **Project structure** — splitting a program into modules by responsibility, and why logic that neither prints nor reads files is far easier to test.
- **argparse sub-commands** — building a real CLI with per-command arguments, automatic help and proper exit codes.
- **Working with JSON** — reading, writing, and handling a file that is missing or corrupt.
- **Dictionaries and defaultdict** — grouping and sorting data without any external library.
- **Input validation** — checking user input before it reaches storage, and raising exceptions the caller can turn into friendly messages.
- **Testing with unittest** — writing tests for both success and failure cases, and using temporary directories so tests

stay isolated.

- **Documentation** — a README that lets someone else run the project without asking any questions.

13. Future scope

1. **Monthly budget limits** with a warning when a category crosses its limit.
2. **Editing** an existing expense instead of deleting and re-adding it.
3. **Search** by text in the note field.
4. **Charts as images** using matplotlib, alongside the text bar chart.
5. **SQLite storage** for faster filtering once the file grows large.
6. **Recurring expenses** such as rent added automatically each month.
7. **A small web or GUI front-end** reusing the same `tracker.py` functions.

14. Conclusion

The project set out to build a complete and genuinely useful Python application within a one-month internship. The result records an expense in a single short command, answers the "where did my money go" question in another, keeps the data in a file the user owns and can read, and refuses bad input with a clear message instead of a crash.

The more valuable outcome was the way it was built: separating the logic from the printing and the file handling, validating input at the boundary, and writing tests that cover the failures as well as the successes. Those habits are the part of this internship that I will carry into every project after it.

15. References

1. Python Software Foundation — *Standard Library Reference* (`json`, `csv`, `argparse`, `datetime`, `collections`, `pathlib`, `unittest`). <https://docs.python.org/3/library/>
2. Python Documentation — *argparse tutorial*. <https://docs.python.org/3/howto/argparse.html>
3. Python Documentation — *unittest — Unit testing framework*. <https://docs.python.org/3/library/unittest.html>
4. PEP 8 — *Style Guide for Python Code*. <https://peps.python.org/pep-0008/>
5. PEP 257 — *Docstring Conventions*. <https://peps.python.org/pep-0257/>

Annexure — Internship credentials

Document	Reference
Internship Offer Letter	Issued 08/07/2026 by Dr. Anurag Shrivastava, Talent Acquisition Manager, Codec Technologies India
Internship Certificate	Awarded to Samvit Srivastava for completing the 1-Month AICTE & ICAC approved internship as Python Developer Intern, 08/07/2026 – 08/08/2026
AICTE ID	CORPORATE6759d549ce59e1733940553
NCS ID	E19E86-0116588288923
Portal	National Internship Portal, Ministry of Education

Project repository: <https://github.com/samvit-iilm/python-Internship> **Submitted:** September 2026